



Natural Language Processing IN ACTION

SECOND EDITION

Hobson Lane
Maria Dyshel

 MANNING

Praise for the First Edition

Learn both the theory and practical skills needed to go beyond merely understanding the inner workings of NLP, and start creating your own algorithms or models.

—Dr. Arwen Griffioen, Zendesk, from the foreword to the first edition

Natural language processing unleashed—Go from novice to ninja!

—Parthasarathy C. Mandayam, senior technical lead at XECOM Information Technologies

A deep dive in natural language processing for human-machine cooperation.

—Simona Russo, technical director at Serendipity S.r.l.

Gives a thorough, in-depth look at natural language processing, starting from the basics, all the way up to state-of-the-art problems.

—Srdjan Santic, data science mentor at Springboard.com

An intuitive guide to start with natural language processing, which also covers deep learning techniques for NLP and real-world use cases. The book is full of many programming examples which help to learn the subject in a very pragmatic way.

—Tommaso Teofili, computer scientist at Adobe Systems

Natural Language Processing in Action provides a great overview of current NLP tools in Python. I'll definitely be keeping this book on hand for my own NLP work. Highly recommended!

—Tony Mullen, associate professor at Northeastern University, Seattle



Natural Language Processing in Action, Second Edition

Hobson Lane and Maria Dyshe

To comment go to [livebook](#).



Manning
Shelter Island

For more information on this and other Manning titles go to [manning.com](#).

copyright

For online information and ordering of this and other Manning books, please visit [www.manning.com](#). The publisher offers discounts on this book when ordered in quantity. For more information, please contact

Special Sales Department

Manning Publications Co.

20 Baldwin Road

PO Box 761

Shelter Island, NY 11964

Email: orders@manning.com

©2025 by Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the

book, and Manning Publications was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The authors and publisher have made every effort to ensure that the information in this book was correct at press time. The authors and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.

Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

Development editor: Karen Miller
Technical editor: Kostas Passadis
Review editor: Dunja Nikitović
Production editor: Keri Hales
Copy editor: Christian Berk
Proofreader: Katie Tennant
Technical proofreaders: Mayur Patil, Kimberly Fessel
Typesetter: Dennis Dalinnik
Cover designer: Marija Tudor

ISBN: 9781617299445

Printed in the United States of America

contents

[*preface*](#)

[*acknowledgments*](#)

[*about this book*](#)

[*about the authors*](#)

[*about the contributors*](#)

[*about the cover illustration*](#)

Part 1 Wordy machines: Vector models of natural language

1 Machines that read and write: A natural language processing overview

1.1 Programming languages vs. NLP

1.1.1 Natural language understanding

1.1.2 Natural language generation

1.1.3 Plumbing it all together for positive-impact AI

1.2 The magic of natural language

1.2.1 Language and thought

1.2.2 Machines that converse

1.2.3 The math

1.3 Applications

1.3.1 Processing programming languages with NLP

1.4 Language through a computer's "eyes"

1.4.1 The language of locks

1.4.2 Regular expressions

1.5 Building a simple chatbot

1.5.1 Keyword-based greeting recognizer

1.5.2 Pattern-based intent recognition

1.5.3 Another way to recognize greetings

1.6 A brief overflight of hyperspace

1.7 Word order and grammar

1.8 A chatbot natural language pipeline

1.9 Processing in depth

1.10 Natural language IQ

1.11 Test yourself

2 Tokens of thought: Natural language words

2.1 Tokens and tokenization

2.1.1 Your tokenizer toolbox

2.1.2 The simplest tokenizer

[2.1.3 Rule-based tokenization](#)

[2.1.4 SpaCy](#)

[2.1.5 Finding the fastest word tokenizer](#)

[2.2 Beyond word tokens](#)

[2.2.1 WordPiece tokenizers](#)

[2.3 Improving your vocabulary](#)

[2.3.1 Extending your vocabulary with n-grams](#)

[2.3.2 Normalizing your vocabulary](#)

[2.4 Challenging tokens: Processing logographic languages](#)

[2.4.1 A complicated picture: Lemmatization and stemming in Chinese](#)

[2.5 Vectors of tokens](#)

[2.5.1 One-hot vectors](#)

[2.5.2 Bag-of-words vectors](#)

[2.5.3 Why not bag of characters?](#)

[2.6 Sentiment](#)

[2.6.1 VADER: A rule-based sentiment analyzer](#)

[2.6.2 Naive Bayes](#)

[2.7 Test yourself](#)

3 Math with words: Term frequency–inverse document frequency vectors

[3.1 Bag-of-words vectors](#)

[3.2 Vectorizing text DataFrame constructor](#)

[3.2.1 Faster, better, easier token counting](#)

[3.2.2 Vectorizing your code](#)

[3.2.3 Vector space TF–IDF \(term frequency–inverse document frequency\)](#)

[3.3 Vector distance and similarity](#)

[3.3.1 Dot product](#)

[3.4 Counting TF–IDF frequencies](#)

[3.4.1 Analyzing “this”](#)

[3.5 Zipf’s law](#)

[3.6 Inverse document frequency](#)

[3.6.1 Return of Zipf](#)

[3.6.2 Relevance ranking](#)

[3.6.3 Smoothing out the math](#)

[3.7 Using TF-IDF for your bot](#)

[3.8 What's next](#)

[3.9 Test yourself](#)

4 Finding meaning in word counts: Semantic analysis

[4.1 From word counts to topic scores](#)

[4.1.1 The limitations of TF-IDF vectors and lemmatization](#)

[4.1.2 Topic vectors](#)

[4.1.3 Thought experiment](#)

[4.1.4 Algorithms for scoring topics](#)

[4.2 The challenge: Detecting toxicity](#)

[4.2.1 Linear discriminant analysis classifier](#)

[4.2.2 Going beyond linear](#)

[4.3 Reducing dimensions](#)

[4.3.1 Enter principal component analysis](#)

[4.3.2 Singular value decomposition](#)

[4.4 Latent semantic analysis](#)

[4.4.1 Diving into semantics analysis](#)

[4.4.2 TruncatedSVD or PCA?](#)

[4.4.3 How well does LSA perform for toxicity detection?](#)

[4.4.4 Other ways to reduce dimensions](#)

[4.5 Latent Dirichlet allocation](#)

[4.5.1 The LDiA idea](#)

[4.5.2 LDiA topic model for comments](#)

[4.5.3 Detecting toxicity with LDiA](#)

[4.5.4 A fairer comparison: 32 LDiA topics](#)

[4.6 Distance and similarity](#)

[4.7 Steering with feedback](#)

[4.8 Topic vector power](#)

[4.8.1 Semantic search](#)

[4.9 Equipping your bot with semantic search](#)

[4.10 Test yourself](#)

[Part 2 Deeper learning: Neural networks](#)

[5 Word brain: Neural networks](#)

[5.1 Why neural networks?](#)

[5.1.1 Neural networks for words](#)

[5.1.2 Neurons as feature engineers](#)

[5.1.3 Biological neurons](#)

[5.1.4 Perceptron](#)

[5.1.5 A Python perceptron](#)

[5.2 An example logistic neuron](#)

[5.2.1 The logistics of clickbait](#)

[5.2.2 Sex education](#)

[5.2.3 Pronouns, gender, and sex](#)

[5.2.4 Sex logistics](#)

[5.2.5 A sleek, new PyTorch neuron](#)

[5.3 Skiing down the error slope](#)

[5.3.1 Off the chair lift, onto the slope: Gradient descent and local minima](#)

[5.3.2 Shaking things up: Stochastic gradient descent](#)

[5.4 Test yourself](#)

[6 Reasoning with word embeddings](#)

[6.1 This is your brain on words](#)

[6.2 Applications](#)

[6.2.1 Search for meaning](#)

[6.2.2 Combining word embeddings](#)

[6.2.3 Analogy questions](#)

[6.2.4 Word2Vec innovation](#)

[6.2.5 Artificial intelligence relies on embeddings](#)

[6.3 Word2Vec](#)

[6.3.1 Analogy reasoning](#)

[6.3.2 Learning word embeddings](#)

[6.3.3 Learning meaning without a dictionary](#)

[6.3.4 Using the gensim.word2vec module](#)

[6.3.5 Generating your own word vector representations](#)

[6.4 Word2Vec alternatives](#)

[6.4.1 GloVe](#)

[6.4.2 fastText](#)

[6.4.3 Word2Vec vs. LSA](#)

[6.4.4 Static vs. contextualized embeddings](#)

[6.4.5 Visualizing word relationships](#)

[6.4.6 Making connections](#)

[6.4.7 Unnatural words](#)

[6.5 Test yourself](#)

[7 Finding kernels of knowledge in text with CNNs](#)

[7.1 Patterns in sequences of words](#)

[7.2 Convolution](#)

[7.2.1 Stencils for natural language text](#)

[7.2.2 A bit more stenciling](#)

[7.2.3 Correlation vs. convolution](#)

[7.2.4 Convolution as a mapping function](#)

[7.2.5 Python convolution example](#)

[7.2.6 PyTorch 1D CNN on 4D embedding vectors](#)

[7.2.7 Natural examples](#)

[7.3 Morse code](#)

[7.3.1 Decoding Morse with convolution](#)

[7.4 Building a CNN with PyTorch](#)

[7.4.1 Clipping and padding](#)

[7.4.2 Better representation with word embeddings](#)

[7.4.3 Transfer learning](#)

[7.4.4 Robustifying your CNN with dropout](#)

[7.5 PyTorch CNN to process disaster toots](#)

[7.5.1 Network architecture](#)

This document was truncated here because it was created in the Evaluation Mode.

